Design Deviations from Original UML Diagrams

Overview

The initial UML diagrams in the docs/ folder outlined a high-level architecture with stubs for networking and database components. The actual implementation introduced several key deviations to support a fully functional multiplayer platform.

1. Server Implementation

Original UML	Implemented
Placeholders for ServerCore and ClientCore with TCP communication outlined	Full server-side logic (PlatformCore as main server class) with complete request routing
Stubbed network layer	Robust server handling client connections and maintaining authoritative game state

What changed: The original diagrams didn't detail how the server would manage lobbies, game sessions, or user data. The implemented system features a complete server-client architecture enabling real-time multiplayer interactions.

2. Database / Persistence System

Original UML	Implemented
Simple stubs like CredentialsDatabase using in-memory HashMaps	File-based JSON storage with domain-driven design principles
Basic file I/O	Thread-safe operations with read-write locks

What changed: The implemented system provides data integrity, scalability beyond the initial stubs, and uses Java Reflections for dynamic type resolution in the base Store class.

3. Additional Components

Component	Original UML	Implemented
Matchmaking	Basic outline	Full service with queuing

		and real-time updates
Chat System	Not specified	Integrated chat in lobbies
Session Management	Not detailed	Ensures one active session per user to prevent hijacking
User Ranking	Basic outline	Game-agnostic score system for consistent ranking across games
Leaderboard	Basic outline	Leaderboard is a sorted list of playerstats and follows a set of rules to determine best players

4. Game Implementation:

Original UML	**Implemented**
Abstract Game interface outlined	**Fully functional games (Tic-Tac-Toe and Connect 4)**
No concrete implementations	**Fully functional games (Tic-Tac-Toe and Connect 4)**

What changed: Each game implements RuleValidator to enforce game-specific rules independently of the UI, and uses PlayerAction classes to model moves and state transitions. This design allows easy extension with new games.

5. Architecture Benefits Realized

The deviations from original UML enabled:

Area	Benefit
Server as source of truth	All data mutations occur server-side, preventing client inconsistencies
Thread safety	Read-write, locks and synchronized functions ensure concurrent access without data corruption
Persistence	Data survives application restarts via JSON files in datafiles/ directory
Normalization	Game history and participant data

	normalized to minimize redundancy
Flexibility	Easy to add new stores or modify schemas without external dependencies